

```
In [73]: from pdf2image import convert_from_path
from IPython.display import display
images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\TES
for img in images:
    display(img)
```

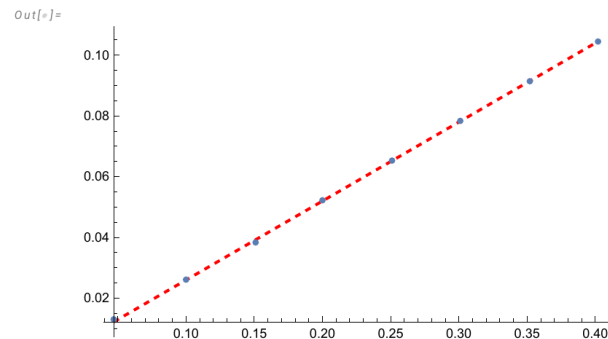
Problem 1.

```
In[*]:= KE = {16, 32, 47, 64, 80, 96, 112, 128} / 1000;
d = {0.047, 0.10, 0.151, 0.200, 0.251, 0.301, 0.352, 0.402};
m = 0.125; g = 9.80;

In[*]:= y =  $\frac{KE}{m * g}$ ;
data = Thread[{d, y}];
dataplot = ListPlot[data];
fit = LinearModelFit[data, X, X, IncludeConstantBasis -> False];
fit["ParameterTable"]
fitplot = Plot[fit[X], {X, Min[d], Max[d]}, PlotStyle -> {Red, Dashed}];
Show[fitplot, dataplot]
```

```
Out[*]:=
```

	Estimate	Standard Error	t-Statistic	P-Value
X	0.259933	0.000666788	389.828	1.93019×10^{-16}

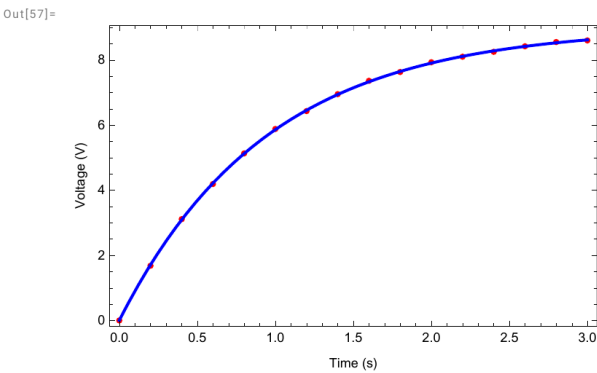


Problem 4.
Part (a)

```
In[51]:= tVals = {0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4, 2.6, 2.8, 3.0};
VcVals = {0.00, 1.68, 3.11, 4.19, 5.13, 5.88,
          6.43, 6.95, 7.36, 7.63, 7.93, 8.10, 8.25, 8.42, 8.55, 8.60};
resistor = 22600;
data = Transpose[{tVals, VcVals}];
nlm = NonlinearModelFit[data, v * (1 - Exp[-t / (resistor * cap)]), {{v, 9.0}, {cap, 0.0001}}, t];
(*results*)
nlm["ParameterTable"]
Show[ListPlot[data, PlotStyle -> Red, Frame -> True,
            FrameLabel -> {"Time (s)", "Voltage (V)"}], Plot[nlm[t], {t, 0, 3}, PlotStyle -> Blue]]
```

Out[56]=

	Estimate	Standard Error	t-Statistic	P-Value
v	8.99831	0.016171	556.447	8.09032×10^{-32}
cap	0.0000420104	2.09302×10^{-7}	200.716	1.27901×10^{-25}



Problem 4(b)

In[122]:=

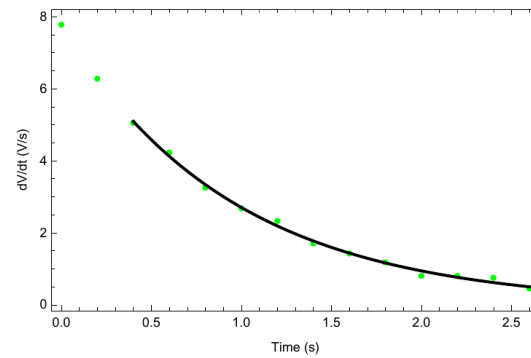
```

current = Table[ $\frac{(VcVals[[i + 2]] - VcVals[[i]])}{2 * 0.2}$ , {i, 1, Length[VcVals] - 2}];
t2 = Table[tVals[[i]], {i, 1, Length[VcVals] - 2}];
idata = Thread[{t2, current}];
ifit = NonlinearModelFit[idata, a * Exp[-k * t], {{a, 10}, {k, 0.5}}, t];
Print[ifit["ParameterTable"]]
Show[ListPlot[idata, PlotStyle -> Green,
  Frame -> True, FrameLabel -> {"Time (s)", "dV/dt (V/s)"},
  Plot[ifit[t], {t, 0.4, 2.6}, PlotStyle -> Black]]

```

	Estimate	Standard Error	t-Statistic	P-Value
a	7.76602	0.0650122	119.455	7.94181×10^{-20}
k	1.05614	0.0143872	73.4084	2.71684×10^{-17}

Out[127]=



Printed by Wolfram Mathematica Student Edition

Problem 2.

```

In [42]: import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

t=np.arange(0.00, 4.25, 0.25)
x=np.array([4.0, 3.5, 2.1, 0.2, -1.6, -2.9, -3.5, -3.1, -2.0,

```

```

-0.6, 0.7, 1.6, 2.0, 1.9, 1.4, 0.9, 0.5])

def f(t, A, omega):
    return A*np.cos(omega*t)

min_index=np.argmin(x)
min_x=min(x)
valley=t[min_index]
tA1=t[:min_index+1]
xA1=x[:min_index+1]

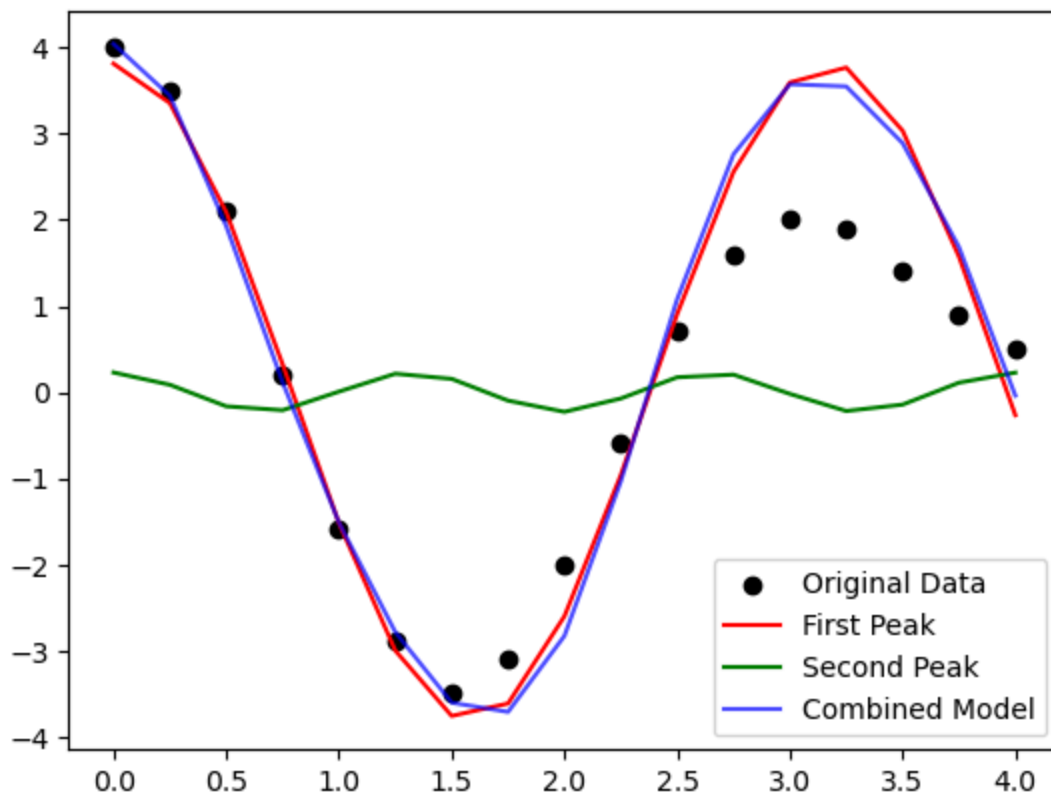
initial_guess=[np.argmax(x), 2]
popt, pcov = curve_fit(f, tA1, xA1, p0 = initial_guess)

xA2= x-f(t, *popt)
initial_guess_A2 = [0.2, 4.7]
popt2, pcov2 = curve_fit(f, t, xA2, p0 = initial_guess_A2)

plt.scatter(t, x, label = "Original Data", color = "black")
plt.plot(t, f(t, *popt), "r", label = "First Peak")
plt.plot(t, f(t, *popt2), "g", label = "Second Peak")
plt.plot(t, f(t, *popt) + f(t, *popt2), "b", label="Combined Model", alpha=0.7)
plt.legend()
plt.show()

print(f"Force 1: A1={popt[0]:.2f}, omega1={popt[1]:.2f}")
print(f"Force 2: A2={popt2[0]:.2f}, omega2={popt2[1]:.2f}")

```



Force 1: A1=3.82, omega1=1.98
Force 2: A2=0.23, omega2=4.74

Problem 3.

(a)

$$\frac{-dQ}{dt} = \sigma\epsilon AT^4; dQ = mcdT; \rho = \frac{m}{V} \Rightarrow m = \rho v$$

$$-mc \frac{dT}{dt} = \sigma\epsilon AT^4$$

$$-\rho \frac{4}{3} \pi r^3 c \frac{dT}{dt} = \sigma\epsilon 4\pi r^2 T^4$$

$$-\rho r c \frac{dT}{dt} = 3\sigma\epsilon T^4$$

$$-\frac{dT}{T^4} = \frac{3\sigma\epsilon}{\rho r c} dt$$

$$-\int_{T_0}^T T^{-4} dT = \frac{3\sigma\epsilon}{\rho r c} \int dt$$

$$\frac{1}{3} (T^{-3} - T_0^{-3}) = \frac{3\sigma\epsilon}{\rho r c} t$$

$$T^{-3} - T_0^{-3} = \frac{9\sigma\epsilon}{\rho r c} t$$

$$T^3 = T_0^3 + \left(\frac{9\sigma\epsilon}{\rho r c} t \right)^{-1}$$

$$T^3 = T_0^3 \left(1 + \frac{9\sigma\epsilon T_0^3}{\rho r c} t \right)^{-1}$$

$$T = T_0 \left(1 + \frac{9\sigma\epsilon T_0^3}{\rho r c} t \right)^{-\frac{1}{3}}$$

(b)

```
In [72]: from matplotlib import pyplot as plt
from scipy.optimize import curve_fit
import numpy as np
```

```

t=np.arange(1.00,9.00,1.00)
t_smooth=np.linspace(1,8.1,100)
T=np.array([1310, 1115, 979, 904, 846, 801, 756, 728])

#define Constants
T0=2000
s=5.6704e-8
rho=7910
r=0.45
c=452

def stefan(t, e):
    t = t*3600
    return T0*(1+9*s*e*T0**3*t/(rho *c *r))**(-1/3)

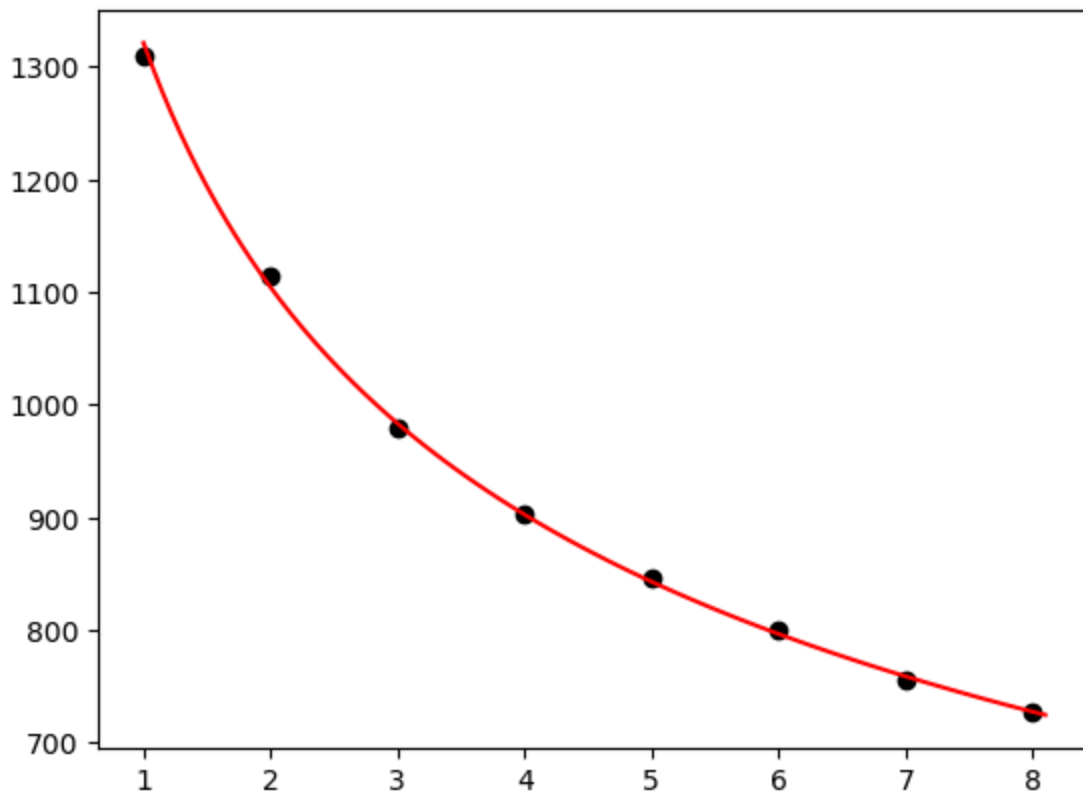
initial_guess=[0.5]
popt, pcov = curve_fit(stefan, t, T, p0 = initial_guess)
perr = np.sqrt(np.diag(pcov))

print(f"Value for Emissivity: {popt[0]:.3f} +/- {perr[0]:.3f}")

plt.scatter(t, T, label = "Original Data", color = "black")
plt.plot(t_smooth, stefan(t_smooth, *popt), "r", label = "Fitted Model")
plt.plot()
plt.show()

```

Value for Emissivity: 0.270 +/- 0.002



In []: